

## Código de ejemplo: Conexión a DB

### Prompt

Utilizando SQL Server y el lenguaje C# con el framework .NET, realice un código para crear una página web, que muestre los resultados de 3 consultas SQL de la base de datos

La conexión a la base de datos, debe ser parametrizable

Las consultas SQL deben ser parametrizables

Cada una de las consultas debe ser un método diferente, utilizando el mismo método para realizar la conexión a la base de datos

En la página web, cada una de las consultas va a tener asociado un botón que va a llamar la consulta específica

Indique el paso a paso, para generar y ejecutar el código correspondiente

El proyecto se debe crear por medio de la línea de comandos

### Pasos

Paso 1: Crear el proyecto ASP.NET Core Web Application

Abre la terminal y ejecuta:

```
dotnet new mvc -n ConsultasWeb  
cd ConsultasWeb
```

Esto crea un proyecto MVC llamado ConsultasWeb.

Paso 1b. Instala el paquete NuGet Microsoft.Data.SqlClient

Desde la línea de comandos, en la raíz del proyecto, ejecuta:

```
dotnet add package Microsoft.Data.SqlClient
```

Esto agregará la referencia necesaria.

Paso 2: Crear una clase para gestionar la conexión y las consultas

Dentro de la carpeta Models, crea un archivo llamado **DatabaseHelper.cs**. Este contendrá métodos para conectarse y realizar las consultas.

Ejemplo de contenido:

```
using System;
using System.Data;
using Microsoft.Data.SqlClient;
using System.Threading.Tasks;

namespace ConsultasWeb.Models
{
    public class DatabaseHelper
    {
        private readonly string _connectionString;

        public DatabaseHelper(string connectionString)
        {
            _connectionString = connectionString;
        }

        private SqlConnection GetConnection()
        {
            return new SqlConnection(_connectionString);
        }

        public async Task<DataTable> EjecutarConsultaAsync(string consultaSQL, params
SqlParameter[] parametros)
        {
            using (var connection = GetConnection())
            {
                using (var command = new SqlCommand(consultaSQL, connection))
                {
                    if (parametros != null)
                        command.Parameters.AddRange(parametros);

                    await connection.OpenAsync();

                    var reader = await command.ExecuteReaderAsync();

                    var dt = new DataTable();
                    dt.Load(reader);

                    return dt;
                }
            }
        }

        // Consulta por edad
        public async Task<DataTable> Consulta1Async(int parametro)
        {
            string sql = "SELECT * FROM Clientes WHERE Edad = @parametro";
```

```

        return await EjecutarConsultaAsync(
            sql,
            new SqlParameter("@parametro", parametro)
        );
    }

    // Consulta por categoría
    public async Task<DataTable> Consulta2Async(string parametro)
    {
        string sql = "SELECT * FROM Productos WHERE Categoria = @parametro";

        return await EjecutarConsultaAsync(
            sql,
            new SqlParameter("@parametro", parametro)
        );
    }

    // Consulta por fecha de venta
    public async Task<DataTable> Consulta3Async(DateTime parametro)
    {
        string sql = "SELECT * FROM Ventas WHERE FechaVenta = @parametro";

        return await EjecutarConsultaAsync(
            sql,
            new SqlParameter("@parametro", parametro)
        );
    }
}

```

Paso 3: Configurar la conexión en appsettings.json

Edita appsettings.json para incluir la cadena de conexión:

```

{
  "ConnectionStrings": {
    "DefaultConnection": "Server=localhost;Database=WebQueriesDB;User
Id=sa;Password=123456;TrustServerCertificate=True;Encrypt=False"
  }
}

```

Reemplaza con tus datos reales, y esta es la base de datos de ejemplo que estaremos usando:

```

-- Crear base de datos
CREATE DATABASE WebQueriesDB;
GO

-- Seleccionar la base de datos
USE WebQueriesDB;
GO

/* =====

```

TABLA: Clientes

Utilizada por Consulta1 (parámetro entero)

===== \*/

CREATE TABLE Clientes (

Id INT IDENTITY(1,1) PRIMARY KEY,

Nombre NVARCHAR(100) NOT NULL,

Edad INT NOT NULL

);

GO

INSERT INTO Clientes (Nombre, Edad)

VALUES

('Ana', 25),

('Carlos', 30),

('Maria', 25),

('Luis', 40),

('Sofia', 35);

GO

/\* =====

TABLA: Productos

Utilizada por Consulta2 (parámetro texto)

===== \*/

CREATE TABLE Productos (

Id INT IDENTITY(1,1) PRIMARY KEY,

Nombre NVARCHAR(100) NOT NULL,

Categoria NVARCHAR(50) NOT NULL

);

GO

INSERT INTO Productos (Nombre, Categoria)

VALUES

('Laptop Dell', 'Tecnologia'),

('Mouse Logitech', 'Tecnologia'),

('Monitor Samsung', 'Tecnologia'),

('Silla Ejecutiva', 'Muebles'),

('Escritorio', 'Muebles');

GO

/\* =====

TABLA: Ventas

Utilizada por Consulta3 (parámetro fecha)

===== \*/

CREATE TABLE Ventas (

Id INT IDENTITY(1,1) PRIMARY KEY,

Cliente NVARCHAR(100) NOT NULL,

FechaVenta DATE NOT NULL,

Monto DECIMAL(10,2) NOT NULL

);

GO

INSERT INTO Ventas (Cliente, FechaVenta, Monto)

VALUES

('Ana', '2025-01-15', 150.00),

```

('Carlos', '2025-02-10', 300.50),
('Maria', '2025-03-20', 220.75),
('Luis', '2025-04-05', 500.00),
('Sofia', '2025-05-12', 175.25);
GO

/* =====
CONSULTAS DE PRUEBA
===== */

-- Consulta 1
SELECT *
FROM Clientes
WHERE Edad = 25;
GO

-- Consulta 2
SELECT *
FROM Productos
WHERE Categoria = 'Tecnologia';
GO

-- Consulta 3
SELECT *
FROM Ventas
WHERE FechaVenta = '2025-02-10';
GO

```

Paso 4: Registrar DatabaseHelper en Startup.cs o Program.cs

Dependiendo de la versión de .NET, en **Program.cs**, agrega:

```

using ConsultasWeb.Models;

builder.Services.AddTransient<DatabaseHelper>(sp =>
    new DatabaseHelper(
        builder.Configuration.GetConnectionString("DefaultConnection")!
    ));

```

Si tu proyecto no se llama ConsultasWeb, reemplaza ConsultasWeb por el namespace correspondiente.

Paso 5: Crear el controlador manualmente

Crear el archivo del controlador llamado **ConsultasController.cs** en la carpeta Controllers del proyecto y luego abre Controllers/ConsultasController.cs y modifica:

```

using System;
using Microsoft.AspNetCore.Mvc;
using ConsultasWeb.Models;
using System.Threading.Tasks;

```

```

namespace ConsultasWeb.Controllers
{
    public class ConsultasController : Controller
    {
        private readonly DatabaseHelper _dbHelper;

        public ConsultasController(DatabaseHelper dbHelper)
        {
            _dbHelper = dbHelper;
        }

        // Mostrar la vista con botones
        public IActionResult Index()
        {
            return View();
        }

        // Acción para consulta 1
        [HttpPost]
        public async Task<IActionResult> Consulta1(int parametro)
        {
            var resultado = await _dbHelper.Consulta1Async(parametro);
            return View("Resultado", resultado);
        }

        // Acción para consulta 2
        [HttpPost]
        public async Task<IActionResult> Consulta2(string parametro)
        {
            var resultado = await _dbHelper.Consulta2Async(parametro);
            return View("Resultado", resultado);
        }

        // Acción para consulta 3
        [HttpPost]
        public async Task<IActionResult> Consulta3(string parametro)
        {
            if (DateTime.TryParse(parametro, out DateTime fecha))
            {
                var resultado = await _dbHelper.Consulta3Async(fecha);
                return View("Resultado", resultado);
            }
            else
            {
                ViewBag.Error = "Fecha inválida";
                return View("Index");
            }
        }
    }
}

```

## Paso 6: Crear las vistas

En la carpeta **Views** debes crear la carpeta **Consultas**, y dentro de Consultas crea el archivo **Index.cshtml** con los siguientes botones y formularios:

```
@{
    ViewData["Title"] = "Consultas";
}

<h2>Consultas de Base de Datos</h2>

<p>
    Esta página permite ejecutar tres consultas sobre la base de datos
    <strong>WebQueriesDB</strong>.
</p>

<hr />

<h3>Consulta 1: Buscar clientes por edad</h3>

<p>
    Ingrese una edad para obtener los clientes que coincidan.
</p>

<p>
    Ejemplos: 25, 30, 35
</p>

<form method="post" asp-action="Consulta1">
    <label>Edad:</label>
    <input type="number" name="parametro" required />
    <button type="submit">Buscar Clientes</button>
</form>

<hr />

<h3>Consulta 2: Buscar productos por categoría</h3>

<p>
    Ingrese una categoría para obtener los productos asociados.
</p>

<p>
    Ejemplos:
    <strong>Tecnología</strong>,
    <strong>Muebles</strong>
</p>

<form method="post" asp-action="Consulta2">
    <label>Categoría:</label>
    <input type="text" name="parametro" required />
    <button type="submit">Buscar Productos</button>
</form>
```

```

<hr />

<h3>Consulta 3: Buscar ventas por fecha</h3>

<p>
    Seleccione una fecha para consultar las ventas registradas.
</p>

<p>
    Ejemplo:
    <strong>2025-02-10</strong>
</p>

<form method="post" asp-action="Consulta3">
    <label>Fecha:</label>
    <input type="date" name="parametro" required />
    <button type="submit">Buscar Ventas</button>
</form>

@if (ViewBag.Error != null)
{
    <p style="color:red;">@ViewBag.Error</p>
}

```

Luego, en la misma carpeta, crea **Resultado.cshtml** para mostrar los resultados con el siguiente código:

```

@model System.Data.DataTable
@{
    ViewData["Title"] = "Resultado";
}

<h2>Resultado de la Consulta</h2>

@if (Model != null && Model.Rows.Count > 0)
{
    <table border="1">
        <thead>
            <tr>
                @foreach (System.Data.DataColumn col in Model.Columns)
                {
                    <th>@col.ColumnName</th>
                }
            </tr>
        </thead>
        <tbody>
            @foreach (System.Data.DataRow row in Model.Rows)
            {
                <tr>
                    @foreach (var item in row.ItemArray)
                    {

```



```
        <td>@item</td>
    }
</tr>
}
</tbody>
</table>
}
else
{
    <p>No hay datos para mostrar.</p>
}
<a href="/Consultas/Index">Volver</a>
```

#### Paso 7: Ejecutar la aplicación

En la terminal, en la carpeta del proyecto, ejecuta:

```
dotnet run
```

Después de ejecutar en la consola, te saldrá algo como "Now listening on: <http://localhost:5286>". Abre tu navegador con la dirección que te da porque podría variar en cada ejecución. Después de encontrar dónde se está ejecutando, debes añadir arriba a la ruta `/consultas` para que quede como <http://localhost:5286/consultas> y ya puedes probar las consultas. Dejaré un repositorio de GitHub realizado por el estudiante Frederic Jasmin con todo el código para ejecutarse directamente: <https://github.com/fredericjasmin/aspnet-web-sql-server-queries-lab>